

BAHASA PEMROGRAMAN INDONESIA

DOM & Event

Membangun

Antarmuka

Modul ketiga dari seri pembelajaran KARSA v0.3.1. Pelajari cara menciptakan elemen DOM, memilih dengan selector, memasang event handler, dan merakit antarmuka interaktif lengkap dengan lifecycle komponen.

● Level Pemula-Menengah

● 28 Jam Belajar

● 10 Bab + Mini-Proyek

● 15+ Latihan

Daftar Isi

Modul 3 — DOM & Event: Membangun Antarmuka KARSA · v0.3.1

PENGANTAR

Pengantar Modul & Mind Map	03
Tujuan, prasyarat, peta 10 bab	

BAB UTAMA

Bab 1 — Pernyataan buat	05
20+ tag alias Indonesia, blok properti vs inline	
Bab 2 — Selector KARSA	09
Sintaks tag#id.kelas[attr], beda vs CSS selector	
Bab 3 — Properti DOM & 6 Jenis ambil	12
13 alias Indonesia, ambil nilai/teks/html/tinggi/lebar/atribut	
Bab 4 — tampilkan — 5 Mode	16
tambahkan, ganti, awalan, sebelum, sesudah	
Bab 5 — Mutasi DOM	19
perbarui, sembunyikan, hapus, kosongkan	
Bab 6 — Event: ketika — 13 Keyword	22
diklik, diketik, disubmit, ditekan, dipasang, dll	
Bab 7 — Self-Reference di ketika	26
Tanpa target di dalam buat/komponen (v0.3.1)	
Bab 8 — Rantai Aksi dengan lalu	28
Chaining sync & async, batasan	
Bab 9 — Lifecycle Hook	30
dipasang, diperbarui (baru v0.3.1), dilepas	
Bab 10 — Mini-Proyek: Todo List	33
Step-by-step dari example/todo-app.html	

LATIHAN & SOLUSI

Latihan Soal (15+ soal bertingkat)	38
5 mudah, 5 sedang, 5 sulit	
Solusi Latihan Lengkap	44
Kode + penjelasan + alternatif + pembahasan error	

LAMPIRAN

Lampiran A — Referensi Cepat	49
Cheat sheet semua keyword Modul 3	
Lampiran B — KARSA vs Vanilla JS DOM API	50
18+ padanan sintaks	
Lampiran C — Tabel 20+ Tag Alias Indonesia	51
ruang, judul, paragraf, gambar, tombol, dll	
Lampiran D — Error Codes DOM & Event	52
E1xxx, E2xxx, E3xxx, E4xxx, E5xxx relevan	

Selamat Datang di Modul 3

Modul 3 membawa Anda dari logika murni (Modul 2) ke ranah antarmuka pengguna. Di sini Anda akan belajar bagaimana KARSA berinteraksi dengan DOM — menciptakan elemen, memilihnya dengan selector, memasang event handler, dan merakit UI interaktif lengkap dengan lifecycle komponen. Inilah lapisan di mana KARSA benar-benar bersinar sebagai DSL untuk web.

KARSA v0.3.1 mendesain ulang pengalaman DOM ini dengan kata kunci berbahasa Indonesia yang jelas: **buat** untuk menciptakan elemen, **tampilkan** untuk memasang ke DOM, **perbarui** untuk mutasi properti, **sembunyikan** / **hapus** / **kosongkan** untuk kontrol visibilitas, dan **ketika** untuk event listener. Ditambah 20+ tag alias Indonesia (ruang, judul, paragraf, gambar, tombol) yang membuat kode UI terbaca seperti kalimat bahasa Indonesia sehari-hari.

1 Prasyarat

Sebelum memulai, pastikan Anda telah menyelesaikan **Modul 1: Dasar KARSA** dan **Modul 2: Logika & Kontrol Alur**, dan memahami: (1) deklarasi **data** / **tetap** / **ubah** dengan type hint; (2) tipe data dasar termasuk array dan objek; (3) sintaks **jika** / **ulangi** / **fungsi**; (4) konsep scope lokal vs global; (5) mode **tampilkan** dasar (kita akan perdalam 5 mode di Bab 4). Tanpa pondasi ini, konsep event handler dan lifecycle komponen akan terasa abstrak.

2 Kompetensi yang Diraih

Setelah menyelesaikan Modul 3, Anda diharapkan mampu: (1) menciptakan elemen DOM dengan **buat** dan 20+ tag alias Indonesia; (2) memilih elemen dengan selector KARSA (**tag#id.kelas[attr]**); (3) membaca dan menulis properti DOM via 13 alias Indonesia; (4) menggunakan 5 mode **tampilkan** dengan tepat; (5) memutasikan DOM dengan **perbarui**, **sembunyikan**, **hapus**, **kosongkan**; (6) memasang event handler dengan 13 keyword event; (7) memanfaatkan self-reference **ketika** tanpa target; (8) merantai aksi dengan **lalu**; (9) menggunakan 3 lifecycle hook komponen; (10) membaca dan mengoreksi error DOM/event umum.



DURASI
28 Jam

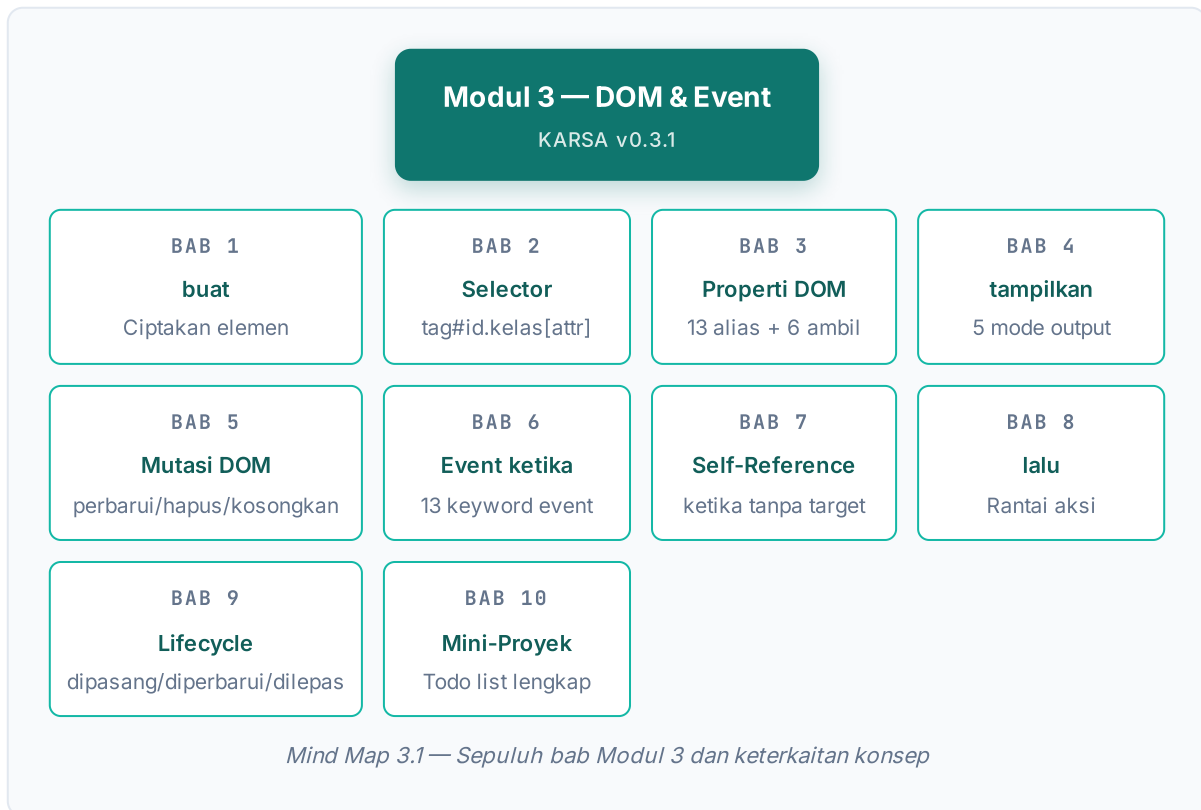


BAB
10 + Proyek



LATIHAN
15+ Soal

3 Peta Konsep Modul 3



Peta di atas menggambarkan alur belajar. Bab 1–5 fokus pada manipulasi DOM statis — menciptakan, memilih, dan memutasinya. Bab 6–8 memperkenalkan interaktivitas via event dan chaining. Bab 9 membahas lifecycle komponen untuk manajemen resource. Bab 10 mengintegrasikan semuanya dalam mini-proyek Todo List yang diambil langsung dari contoh aplikasi di repo dazep01.

BAB 1

Pernyataan **buat** — Menciptakan Elemen

Pernyataan **buat** adalah jantung dari pembangunan UI di KARSA. Dengan satu kata kunci, Anda menciptakan elemen DOM, memberinya ID dan kelas, mengisi properti, dan menambahkan child — semua dalam blok sintaks yang terbaca alami. KARSA menyediakan 20+ tag alias berbahasa Indonesia sehingga kode UI terasa seperti menulis kalimat, bukan memanggil API JavaScript.

1.1 Sintaks Dasar

buat mengikuti pola: kata kunci **buat**, diikuti selector (tag + opsional ID + kelas), lalu opsional blok properti atau properti inline dengan **→**. Elemen yang dibuat otomatis di-**appendChild** ke parent terdekat dalam AST, atau ke **document.body** jika top-level.

```
KARSA · 01-BUAT-DASAR.KS

--! Buat div dengan ID "app" dan kelas "container"
buat div#app.container
  buat h1#judul → teks: "Selamat Datang"
  buat p.deskripsi → teks: "Aplikasi KARSA pertamaku."
```

Kode di atas menghasilkan struktur HTML setara dengan:

```
HTML · OUTPUT KOMPILASI

<div id="app" class="container">
  <h1 id="judul">Selamat Datang</h1>
  <p class="deskripsi">Aplikasi KARSA pertamaku.</p>
</div>
```

1.2 20+ Tag Alias Indonesia

KARSA menyediakan lebih dari 20 alias berbahasa Indonesia untuk tag HTML umum. Alias ini dipakai sebagai pengganti tag HTML native — keduanya valid, tetapi alias Indonesia membuat kode lebih konsisten dengan filosofi KARSA. Berikut tabel lengkapnya.

ALIAS INDONESIA	TAG HTML	KEGUNAAN
ruang	div	Wadah generik blok
judul	h1	Judul utama halaman
subjudul	h2	Sub-judul section
paragraf	p	Paragraf teks
gambar	img	Gambar (perlu src)
tautan	a	Hyperlink (perlu href)
masukan	input	Field input form
kolom	textarea	Input teks multi-baris
wadah	div	Sinonim ruang, untuk variasi semantik
wadjud	div	Wadah khusus (custom)
kotak	div	Wadah bergaya "box"
tombol	button	Tombol yang dapat diklik
frm	form	Formulir
frmMasuk	form	Form khusus login
tabel	table	Tabel data
artikel	article	Konten artikel mandiri
kanvas	canvas	Kanvas untuk gambar 2D
opsi	option	Pilihan dalam dropdown
pilihan	select	Dropdown pilihan
fragmen	DocumentFragment	Wadah virtual, tidak render langsung

● KAPAN PAKAI ALIAS VS NATIVE

Alias Indonesia dianjurkan untuk konsistensi estetika KARSA — kode terbaca lebih alami. Tapi tag HTML native (`div` , `span` , `h1` , dll) tetap valid dan berguna ketika: (1) berkolaborasi dengan developer yang lebih familiar HTML; (2) menggunakan CSS framework yang mengharapkan tag spesifik; (3) aksesibilitas — beberapa screen reader memperlakukan `<button>` berbeda dari `<div>` . Pilih yang paling jelas intent-nya.

1.3 Blok Properti vs Properti Inline

Ada dua cara mengisi properti elemen yang baru dibuat: **properti inline** (satu baris dengan `→`) dan **blok properti** (indentasi, banyak baris). Pilih berdasarkan kompleksitas — inline

untuk 1-2 properti sederhana, blok untuk elemen dengan banyak properti atau child.

Properti Inline

```
buat tombol#btn → teks: "Klik"

buat gambar.avatar → src: "/me.jpg", alt: "Foto"
```

Blok Properti

```
buat div#kartu.profil
  teks: "Profil"
  kelas: "aktif"

  buat h2 → teks: "Nama"
  buat paragraf → teks: "Bio..."
```

1.4 Kunci Properti yang Didukung

Berikut daftar kunci properti yang dapat diisi saat `buat` atau `perbarui`. Kunci ini sudah dikenal parser dan dipetakan ke properti/atribut DOM yang sesuai.

KUNCI	PADANAN DOM	CONTOH
teks	innerText	teks: "HaLo"
html	innerHTML	html: "tebal"
nilai	.value	nilai: "default"
kelas	className	kelas: "aktif tombol"
gaya	style.cssText	gaya: "color: red;"
src	.src	src: "/gambar.jpg"
href	.href	href: "https:// ..."
alt	.alt	alt: "Deskripsi gambar"
tipe	.type	tipe: "email"
nama	.name	nama: "username"
placeholder	.placeholder	placeholder: "Ketik..."
id	.id	id: "kartu-1"
title	.title	title: "Tooltip"
data-*	dataset	data-id: 123

1.5 Contoh Realistik dari Repo

Berikut potongan dari `example/index.ks` di repo dazep01 — aplikasi counter reaktif lengkap dengan struktur nested div + h1 + counter-box:


```

buat div.app-container
  buat h1#judul → teks: pesan_selamat

  buat div.counter-box
    buat p#nilai-text.text-bold → teks: "Nilai saat ini: " + counter
    buat p#status-text → teks: "Status angka: " + status_counter

    buat tombol#btn-tambah → teks: "Tambah Angka"
      ketika diklik → tambahkan 1 ke counter

    buat tombol#btn-reset → teks: "Reset"
      ketika diklik → simpan 0 ke counter

```

● PERHATIKAN INDENTASI

Indentasi 2 spasi menentukan parent-child relationship. Di contoh di atas, `buat p#nilai-text` dan `buat tombol#btn-tambah` sama-sama di-indent 4 spasi (2 level) — keduanya child dari `div.counter-box`. Handler `ketika diklik` di-indent 6 spasi (3 level) — child dari `tombol#btn-tambah`. Indentasi salah = struktur DOM salah.

◆ CATATAN PENGAJAR — BAB 1

DURASI 3 JAM · MATERI + LATIHAN

TUJUAN BELAJAR

Peserta mampu menciptakan struktur DOM bertingkat dengan buat, menggunakan 20+ tag alias Indonesia, dan memilih antara properti inline vs blok.

SARAN MENGAJAR

Mulai dengan analogi "menulis HTML dengan bahasa Indonesia". Tunjukkan kode KARSA lalu hasil HTML-nya berdampingan. Beri latihan: konversi 3 struktur HTML sederhana ke KARSA.

KESALAHAN UMUM

- Lupa bahwa indentasi menentukan parent-child
- Campur tag alias dengan HTML native secara inconsistent
- Pakai `=` bukan `:` untuk properti
- Lupa koma antar properti inline

TROUBLESHOOTING

- **E2001:** Token tidak sesuai — cek selector
- **E2010:** Keyword tidak dikenali — pastikan tag valid
- **E2015:** Selector CSS tidak valid
- **E2026:** Properti kosong

PERTANYAAN DISKUSI

1. Kapan pakai ruang vs div? 2. Mengapa fragmen berguna? 3. Apa efek menggunakan data-id di buat?

Selector KARSA

Selector adalah cara KARSA mengidentifikasi elemen DOM — baik untuk dijadikan target operasi (seperti `tampilkan di`, `perbarui`, `hapus`) maupun sebagai subject event handler. Berbeda dari CSS selector yang kompleks (descendant, sibling, pseudo-class), selector KARSA sengaja dibatasi pada pola tunggal yang jelas: `tag#id.kelas[atribut]`.

2.1 Sintaks Selector

Selector KARSA selalu diawali dengan tag (HTML native atau alias Indonesia), diikuti opsional ID dengan `#`, kelas dengan `.`, dan atribut dengan `[...]`. Urutan ini wajib — tidak boleh `#id tag` atau `.class #id`.

```
KARSA · 03-SELECTOR-SINTAKS.KS

--! Selector dasar: tag saja
buat div

--! Tag + ID
buat div#app

--! Tag + kelas
buat div.container

--! Tag + ID + kelas
buat tombol#btn-submit.primary.large

--! Tag + atribut
buat masukan[type="email"]

--! Lengkap: tag + ID + kelas + atribut
buat masukan#email.form-control[required, placeholder="Email"]
```

2.2 Selector sebagai Target Operasi

Selector juga dipakai sebagai target untuk operasi DOM: `tampilkan ... di <selector>`, `perbarui teks <selector> → ...`, `sembunyikan <selector>`, `hapus <selector>`, `kosongkan <selector>`. Saat dipakai sebagai target, selector ditulis sebagai string literal (dalam tanda kutip) atau identifier (tanpa kutip).

```
KARSA · 04-SELECTOR-TARGET.KS

--! Selector sebagai string literal (umum)
perbarui teks "#angka" → hitungan
sembunyikan ".loading"
kosongkan "#list-tugas"

--! Selector sebagai tag#id.kelas langsung (di blok buat)
buat div#app
  buat h1#judul → teks: "Halo"

--! Selector sebagai identifier (variabel yang menyimpan string selector)
tetap target = "#kartu-1"
sembunyikan target
```

2.3 Beda Selector KARSA vs CSS Selector

ASPEK	SELECTOR KARSA	CSS SELECTOR
Struktur	tag#id.kelas[attr] saja	Compounded: descendant, sibling, pseudo
Descendant	Tidak didukung — pakai blok buat	div > p, div p
Pseudo-class	Tidak didukung	:hover, :nth-child(n)
Universal	Tidak didukung	*
Tujuan	Identifikasi elemen tunggal	Seleksi banyak elemen untuk styling
Hasil	Satu node (atau null)	NodeList (0 atau lebih)

● MENGAPA SENGAJA DIBATASI?

KARSA sengaja tidak mendukung selector kompleks karena: (1) **keterbacaan** — selector tunggal jelas intent-nya; (2) **performa** — querySelector untuk selector kompleks lebih lambat; (3) **prediktabilitas** — selector tunggal selalu return 0 atau 1 node, bukan NodeList ambigu. Untuk operasi multi-node, KARSA menyediakan `ulangi` dengan array yang dikumpulkan eksplisit.

2.4 Selector untuk Event Handler

Di event handler `ketika`, selector adalah target yang akan dipasang listener. Format: `ketika <selector> <event>:` atau `ketika <selector> <event> → <aksi>`. Selector di sini bisa tag, tag#id, atau string literal.

```
--! Event listener di elemen yang sudah ada di DOM
ketika #btn-submit diklik:
    tampilkan pesan "Ter kirim!"

--! Bisa juga pakai tag#id langsung
ketika tombol#btn-hapus diklik → hapus #kartu-1

--! Self-reference (lihat Bab 7) – tanpa target
buat tombol#btn → teks: "Klik"
    ketika diklik → tampilkan pesan "Halo!"
```

● SELECTOR TIDAK DITEMUKAN

Jika selector tidak match dengan elemen apapun di DOM saat runtime, operasi akan **fail silently** — tidak ada error, tapi tidak ada efek. Ini disengaja agar kode tidak crash ketika elemen muncul/menghilang secara dinamis. Untuk debugging, gunakan `karsa check` untuk validasi static, dan `karsa inspect` untuk lihat elemen yang aktif.

◆ CATATAN PENGAJAR — BAB 2

DURASI 2.5 JAM · TEORI + LATIHAN

TUJUAN BELAJAR

Peserta memahami sintaks selector KARSA, perbedaan dengan CSS selector, dan dapat memilih format yang tepat (literal vs identifier vs inline).

SARAN MENGAJAR

Buat tabel perbandingan KARSA vs CSS selector di papan. Beri 5 selector CSS, minta siswa konversi ke KARSA (beberapa tidak bisa — diskusikan alternatifnya).

KESALAHAN UMUM

- Mencoba selector descendant `div p` (tidak didukung)
- Lupa kutip string `"#id"` di target operasi
- Pakai pseudo-class `:hover` di selector KARSA
- Salah urutan `.class#id` (harus `tag#id.class`)

TROUBLESHOOTING

- **E1009**: Selector CSS tidak valid (lexer)
- **E2002**: Selector tidak valid (parser)
- **E2015**: Selector CSS tidak valid (parser lanjut)
- **E2017**: Target event tidak valid

PERTANYAAN DISKUSI

1. Bagaimana cara KARSA menyeleksi banyak elemen sekaligus? 2. Apa risiko "fail silently" ketika selector tidak match? 3. Kapan pakai string vs identifier vs inline?

Properti DOM Lengkap & 6 Jenis `ambil`

Setelah elemen dibuat dan dipasang, Anda perlu membaca dan menulis propertinya — teks yang ditampilkan, nilai input, kelas CSS, atribut data, dan lainnya. KARSA menyediakan 13 alias properti berbahasa Indonesia yang dipetakan ke properti/atribut DOM standar. Untuk membaca, ada 6 jenis `ambil` yang melayani kebutuhan berbeda.

3.1 13 Alias Properti Indonesia

Berikut tabel lengkap alias properti Indonesia. Alias ini dipakai di `perbarui`, di blok properti `buat`, dan sebagai akses properti dot (`elem.teks`, `arr.panjang`). Mereka di-resolve saat fase Resolver (bukan Lexer) untuk fleksibilitas maksimal.

ALIAS KARSA	PADANAN JAVASCRIPT	TIPE	CONTOH
<code>.panjang</code>	<code>.length</code>	Read-only	<code>arr.panjang</code> , <code>teks.panjang</code>
<code>.nilai</code>	<code>.value</code>	Read/write	<code>input.nilai</code>
<code>.teks</code>	<code>.innerText</code>	Read/write	<code>p.teks</code>
<code>.html</code>	<code>.innerHTML</code>	Read/write	<code>div.html</code>
<code>.tipe</code>	<code>.type</code>	Read/write	<code>input.tipe</code>
<code>.nama</code>	<code>.name</code>	Read/write	<code>input.nama</code>
<code>.ditandai</code>	<code>.checked</code>	Read/write	<code>checkbox.ditandai</code>
<code>.nonaktif</code>	<code>.disabled</code>	Read/write	<code>tombol.nonaktif</code>
<code>.indeks</code>	virtual index	Read-only	<code>item.indeks</code> (di loop)
<code>.anak</code>	<code>.children</code>	Read-only	<code>div.anak</code>
<code>.induk</code>	<code>.parentElement</code>	Read-only	<code>elem.induk</code>
<code>.fokus</code>	<code>.focus()</code>	Method/aksi	<code>input.fokus</code>
<code>.atribut</code>	<code>.getAttribute()</code>	Method	<code>elem.atribut("data-id")</code>

● **PERHATIKAN: .FOKUS & .ATRIBUT ADALAH METHOD**

Alias `.fokus` dan `.atribut` sebenarnya adalah *method* (fungsi), bukan properti murni. `.fokus` memanggil `.focus()` untuk memberi fokus keyboard. `.atribut("x")` memanggil `.getAttribute("x")` untuk membaca atribut. Jangan lupa tanda kurung untuk `.atribut`.

3.2 6 Jenis ambil — Membaca dari DOM

Untuk membaca nilai dari DOM ke variabel KARSA, gunakan `ambil <jenis> dari <sumber> → simpan ke <variabel>`. Ada 6 jenis `ambil` yang melayani kebutuhan berbeda.

JENIS	PADANAN DOM	CONTOH PENGGUNAAN
nilai	<code>.value</code>	Baca input form: <code>ambil nilai dari #email → simpan ke email</code>
teks	<code>.innerText</code>	Baca teks elemen: <code>ambil teks dari #judul → simpan ke judul</code>
html	<code>.innerHTML</code>	Baca HTML mentah: <code>ambil html dari #kartu → simpan ke html_kartu</code>
tinggi	<code>.offsetHeight</code>	Ukuran layout box: <code>ambil tinggi dari #panel → simpan ke h</code>
lebar	<code>.offsetWidth</code>	Ukuran layout box: <code>ambil lebar dari #panel → simpan ke w</code>
atribut "x"	<code>.getAttribute("x")</code>	Baca atribut: <code>ambil atribut "data-id" dari #item → simpan ke id</code>

3.3 Contoh Lengkap — Form Input

```
KARSA · 06-FORM-AMBIL.KS

--! Form login sederhana
buat frm#login
  buat masukan#email → tipe: "email", placeholder: "Email"
  buat masukan#password → tipe: "password", placeholder: "Password"
  buat tombol#btn-login → teks: "Login"
  ketika diklik:
    --! Baca nilai input
    ambil nilai dari #email → simpan ke email
    ambil nilai dari #password → simpan ke password

    --! Baca atribut data-*
    ambil atribut "data-csrf" dari #login → simpan ke csrf

    --! Validasi
    jika email == "":
      tampilkan pesan "Email wajib diisi."
      berhenti

    tampilkan pesan "Login: " + email
```

3.4 Akses Properti via Dot Notation

Selain `ambil`, Anda dapat mengakses properti langsung dengan dot notation pada identifier yang menyimpan referensi elemen. Tapi hati-hati — KARSA tidak menyediakan cara native untuk menyimpan referensi elemen ke variabel. Cara idiomatis adalah dengan `ambil` atau via `langsung:` untuk kasus advanced.

```
KARSA · 07-DOT-NOTATION.KS

--! .panjang di array (paling umum)
data daftar = [1, 2, 3, 4]
tampilkan pesan "Panjang: " + daftar.panjang

--! .indeks virtual di loop
ulangi item dari daftar:
  tampilkan pesan "#" + item.indeks + ": " + item

--! Untuk akses elemen kompleks, gunakan langsung:
langsung:
  const btn = document.querySelector("#btn");
  btn.disabled = true;
  btn.focus();
```

● CATATAN: .PANJANG & .INDEKS

Dua alias yang paling sering dipakai adalah `.panjang` (untuk array/string) dan `.indeks` (untuk iterasi `ulang`). Keduanya sudah dibahas di Modul 2 Bab 4. Alias elemen DOM (`.teks`, `.nilai`, dll) lebih sering dipakai via `perbarui` dan `ambil` daripada dot notation langsung.

◆ CATATAN PENGAJAR — BAB 3

DURASI 3 JAM • TEORI + LATIHAN

TUJUAN BELAJAR

Peserta menguasai 13 alias properti Indonesia dan 6 jenis `ambil`, serta dapat memilih cara yang tepat untuk membaca/menulis properti DOM.

SARAN MENGAJAR

Demo form input interaktif: ambil email + password, validasi, tampilkan hasil. Tunjukkan beda teks vs html dengan contoh XSS sederhana (tampilkan input user sebagai html).

KESALAHAN UMUM

- Lupa `→ simpan ke` di akhir `ambil`
- Tukar `teks` dengan `html` (前者 aman,后者 bisa XSS)
- Pakai `.fokus` tanpa tanda kurung (sebenarnya method)
- Lupa `"data-x"` di `ambil atribut` butuh kutip

TROUBLESHOOTING

- **E2024:** `ambil` tanpa konteks jelas
- **E2027:** Properti `perbarui` tidak dikenali
- **E4008:** Properti `perbarui` tidak didukung

PERTANYAAN DISKUSI

1. Kapan harus pakai `ambil html` vs `ambil teks`? 2. Mengapa `.panjang` tidak ada di `ambil`? 3. Bagaimana cara baca `data-id` dari elemen?

BAB 4

Output: `tampilkan` — 5 Mode

Pernyataan `tampilkan` memasang elemen ke DOM atau menampilkan pesan ke pengguna. KARSA v0.3.1 mendukung 5 mode berbeda untuk mengontrol *bagaimana* dan *di mana* konten ditampilkan: `tambahkan` (append), `ganti` (replace), `awalan` (prepend), `sebelum` (insert before sibling), dan `sesudah` (insert after sibling). Memilih mode yang tepat membuat kode UI lebih ekspresif.

4.1 Target `tampilkan`

`tampilkan` menerima beberapa jenis target: selector, literal teks, identifier, dan tiga kategori khusus untuk pesan pengguna.

TARGET	SINTAKS	HASIL
Selector	tampilkan div#kartu di "#app"	Pasang elemen ke kontainer
Literal teks	tampilkan "Hello" di "#out"	Teks langsung sebagai konten
Identifier	tampilkan pesan_selamat di "#judul"	Nilai variabel sebagai konten
Pesan	tampilkan pesan "Tersimpan!"	Notifikasi info biasa
Pesan error	tampilkan pesan-error "Gagal!"	Notifikasi gaya error
Notifikasi	tampilkan notifikasi "Selamat!"	Toast notification

4.2 5 Mode Output

MODE	PADANAN DOM	DEFAULT?	USE CASE
tambahkan	parent.appendChild(el)	Ya (default)	Tambah elemen baru di akhir kontainer
ganti	parent.replaceChildren(el)	Tidak	Bersihkan kontainer & isi dengan elemen baru
awalan	parent.prepend(el)	Tidak	Tambah elemen di awal kontainer
sebelum	target.before(el)	Tidak	Sisipkan sebelum sibling target
sesudah	target.after(el)	Tidak	Sisipkan setelah sibling target

4.3 Contoh Setiap Mode

```
KARSA · 08-TAMPILKAN-MODE.KS

--! Mode "tambahkan" (default) - append ke container
buat div.item → teks: "Item baru"
tampilkan div.item di "#list"
--! atau eksplisit: dengan mode: "tambahkan"

--! Mode "ganti" - bersihkan container, isi baru
buat div#profil → teks: "Profil baru"
tampilkan div#profil di "#app" dengan mode: "ganti"

--! Mode "awalan" - prepend ke container
buat div.badge → teks: "BARU"
tampilkan div.badge di "#list" dengan mode: "awalan"

--! Mode "sebelum" - insert sebelum sibling
buat div.banner → teks: "Promo!"
tampilkan div.banner di "#konten-utama" dengan mode: "sebelum"

--! Mode "sesudah" - insert setelah sibling
buat div.iklan → teks: "Iklan"
tampilkan div.iklan di "#konten-utama" dengan mode: "sesudah"
```

4.4 Notifikasi & Pesan

Untuk komunikasi singkat ke pengguna (bukan struktur DOM), KARSA menyediakan tiga jenis pesan dengan perilaku visual berbeda.

```
KARSA · 09-PESAN.KS

--! pesan - info biasa (console + UI)
tampilkan pesan "Data berhasil disimpan!"

--! pesan-error - error merah
tampilkan pesan-error "Gagal memuat data."

--! notifikasi - toast yang hilang otomatis
tampilkan notifikasi "Tersimpan ✓"
```

● KAPAN PAKAI APA

pesan: untuk informasi penting yang perlu pengguna baca. **pesan-error**: untuk error yang perlu perhatian khusus (gagal fetch, validasi). **notifikasi**: untuk feedback singkat yang tidak boleh mengganggu (toast yang auto-dismiss). Hindari pakai `pesan` untuk hal sepele — akan memenuhi UI.

TUJUAN BELAJAR

Peserta menguasai 5 mode tampilkan dan 3 jenis pesan, serta dapat memilih yang tepat untuk setiap skenario.

SARAN MENGAJAR

Demo visual: pasang elemen ke list dengan 5 mode berbeda, tunjukkan hasil di browser. Lalu demo 3 jenis pesan dengan timing berbeda.

KESALAHAN UMUM

- Lupa `di <target>` — elemen tidak terpasang
- Pakai `tambahkan` padahal butuh `ganti` → duplikat
- Tukar `sebelum` vs `sesudah`
- Pakai `pesan` untuk hal sepele → spam UI

TROUBLESHOOTING

- **E2022**: Target `tampilkan` tidak valid
- **E4007**: Mode tidak valid — pakai 5 mode resmi
- Elemen tidak muncul → cek `di <target>`

PERTANYAAN DISKUSI

1. Apa beda ganti vs kosongkan lalu tambahkan? 2. Kapan pakai notifikasi vs pesan? 3. Bagaimana cara render list 100 item efisien?

BAB 5

Mutasi DOM: perbarui, sembunyikan, hapus, kosongkan

Setelah elemen ada di DOM, Anda perlu empat operasi mutasi: `perbarui` untuk mengubah properti (teks, kelas, src), `sembunyikan` untuk menyembunyikan tanpa menghapus, `hapus` untuk menghilangkan dari DOM, dan `kosongkan` untuk menghapus semua child tanpa menghapus elemen itu sendiri. Empat operasi ini mencakup 90% kebutuhan mutasi DOM sehari-hari.

5.1 perbarui — Update Properti Elemen

```
KARSA · 10-PERBARUI.KS

--! Update teks elemen
perbarui teks "#angka" → hitungan

--! Update html (hati-hati XSS)
perbarui html "#konten" → "<b>Tebal</b>"

--! Update nilai input
perbarui nilai #nama → ""

--! Update kelas (replace, bukan merge)
perbarui kelas "#kotak" → "aktif"

--! Update atribut gaya (style)
perbarui gaya "#panel" → "background: red; color: white;"
```

● PERBARUI KELAS = REPLACE, BUKAN MERGE

`perbarui kelas <target> → "aktif"` akan **mengganti seluruh** string className elemen dengan "aktif". Kelas lama hilang. Jika ingin *menambah* kelas tanpa menghapus yang lain, gunakan `langsung:` dengan `classList.add("aktif")`.

5.2 sembunyikan — Sembunyikan Tanpa Hapus

`sembunyikan` mengeset `style.display = "none"` — elemen tidak terlihat dan tidak mengambil ruang layout, tapi tetap ada di DOM. Berguna untuk toggle visibility tanpa kehilangan state atau posisi.

```
KARSA · 11-SEMBUNYIKAN.KS

--! Sembunyikan loading spinner
sembunyikan "#loading"

--! Toggle visibility berdasarkan state
jika sedang_memuat:
  --! Tampilkan loading (butuh langsung: untuk display: block)
  langsung:
    document.querySelector("#loading").style.display = "block";
jika tidak:
  sembunyikan "#loading"
```

5.3 hapus — Hapus dari DOM

`hapus` memanggil `element.remove()` — elemen dihilangkan dari DOM sepenuhnya. Berbeda dari `sembunyikan`, elemen tidak bisa "ditampilkan lagi" karena sudah tidak ada. Event listener yang terikat juga ikut terbang.

```
KARSA · 12-HAPUS.KS

--! Hapus item dari daftar
hapus "#item-3"

--! Hapus modal setelah ditutup
ketika #btn-tutup diklik:
  hapus "#modal"
```

5.4 kosongkan — Bersihkan Child

`kosongkan` memanggil `replaceChildren()` — menghapus semua child elemen, tapi elemen itu sendiri tetap ada. Sangat berguna untuk merender ulang list: kosongkan kontainer, lalu `ulangi` untuk tambah item baru.

```
KARSA · 13-KOSONGKAN.KS (DARI TODO-APP.HTML)

--! Render ulang daftar tugas setiap kali data berubah
saat semuaTugas berubah:
  --! Bersihkan kontainer list
  kosongkan #list-tugas

  --! Tambah ulang semua item
  ulangi tugas dari semuaTugas:
    buat li
      buat span → teks: tugas
      buat tombol.hapus-btn → teks: "Hapus"

  --! Update counter
  perbarui teks #total-angka → semuaTugas.panjang
```

5.5 Perbandingan 4 Mutator

OPERASI	PADANAN DOM	EFEK	USE CASE
perbarui	<code>.innerText = dll</code>	Ubah properti, elemen tetap	Update teks/nilai/kelas
sembunyikan	<code>.style.display = "none"</code>	Tidak terlihat, tetap di DOM	Toggle visibility
hapus	<code>.remove()</code>	Hilang dari DOM sepenuhnya	Hapus item, tutup modal
kosongkan	<code>.replaceChildren()</code>	Hapus semua child, elemen tetap	Render ulang list

◆ CATATAN PENGAJAR — BAB 5

DURASI 2.5 JAM • TEORI + PRAKTIK

TUJUAN BELAJAR

Peserta menguasai 4 mutator DOM dan dapat memilih yang tepat: perbarui untuk properti, sembunyikan untuk toggle, hapus untuk hilangkan, kosongkan untuk bersihkan child.

SARAN MENGAJAR

Demo todo app: tambah item (perbarui counter), hapus item (hapus), toggle filter (sembunyikan), refresh list (kosongkan + ulangi). Tunjukkan kapan masing-masing dipakai.

KESALAHAN UMUM

- Harap `perbarui kelas` merge, padahal replace
- Pakai `hapus` padahal butuh `sembunyikan` (kehilangan state)
- Lupa `kosongkan` sebelum render ulang list → duplikat
- Pakai `perbarui html` dengan input user → XSS

TROUBLESHOOTING

- **E2016:** Token → diharapkan di `perbarui`
- **E2027:** Properti `perbarui` tidak dikenali
- **E4008:** Properti tidak didukung analyzer

PERTANYAAN DISKUSI

1. Apa beda hapus `#list` vs kosongkan `#list`? 2. Bagaimana cara aman update kelas tanpa hilang yang lama? 3. Mengapa `perbarui html` berbahaya?

BAB 6

Event: **ketika** — 13 Keyword Event

Interaktivitas lahir dari event — klik tombol, ketik input, submit form, scroll halaman. KARSA v0.3.1 menyediakan 13 keyword event berbahasa Indonesia yang dipetakan ke event DOM standar. Cukup tulis `ketika <target> <event>:` dan KARSA akan memasang `addEventListener` dengan binding yang benar.

6.1 13 Keyword Event

KEYWORD KARSA	EVENT JAVASCRIPT	KAPAN DIPICU
diklik	click	Klik kiri mouse pada elemen
diketik	input	Setiap karakter diketik di input/textarea
disubmit	submit	Form di-submit (tombol atau Enter)
dimuat	load	Elemen/halaman selesai dimuat
diubah	change	Nilai berubah & kehilangan fokus (select, checkbox)
difokus	focus	Elemen mendapat fokus keyboard
ditinggal	blur	Elemen kehilangan fokus
ditekan	keydown	Tombol keyboard ditekan
dilepas	keyup	Tombol keyboard dilepas
diarahkan	mouseenter	Kursor masuk ke area elemen
ditinggal-kursor	mouseleave	Kursor keluar dari area elemen
digulir	scroll	Pengguna scroll halaman/elemen
dipasang / dilepas-dari-dom	lifecycle	Elemen ditambahkan/dihapus dari DOM

6.2 Sintaks ketika

Ada dua bentuk: **blok aksi** (multi-aksi dengan indentasi) dan **aksi tunggal** (satu baris dengan `→`). Target bisa selector atau dihilangkan (self-reference, lihat Bab 7).

```
KARSA · 14-KETIKA-SINTAKS.KS

--! Bentuk blok aksi (multi-aksi)
ketika #btn-submit diklik:
  ambil nilai dari #email → simpan ke email
  jika email == "":
    tampilkan pesan-error "Email wajib diisi."
    berhenti
  tampilkan pesan "Terkirim!"

--! Bentuk aksi tunggal (satu baris)
ketika #btn-reset diklik → simpan 0 ke counter
```

6.3 Aturan Khusus disubmit

Secara default, `ketika form disubmit:` akan menyisipkan `preventDefault()` — form tidak melakukan reload halaman. Ini karena mayoritas aplikasi modern memproses submit via JavaScript. Jika Anda ingin perilaku bawaan browser (reload), gunakan opsi eksplisit.

```
KARSA · 15-DISUBMIT.KS

--! Default: preventDefault aktif (tidak reload)
ketika frm#login disubmit:
  ambil nilai dari #email → simpan ke email
  tampilkan pesan "Login: " + email

--! Override: izinkan reload (kasus langka)
ketika frm#legacy disubmit dengan izinkan-default: benar:
  --! Form akan reload seperti HTML biasa
  tampilkan pesan "Mengirim ke server..."
```

6.4 Beda diketik vs ditekan vs diubah

Tiga event yang sering tertukar. Pahami bedanya untuk memilih yang tepat.

EVENT	PEMICU	FREKUENSI	USE CASE
diketik	Setiap karakter berubah	Sangat sering	Validasi real-time, live search
ditekan	Tombol fisik ditekan	Sering	Shortcut keyboard (Enter, Esc)
diubah	Nilai berubah + blur	Jarang (sekali per perubahan)	Validasi saat selesai edit


```

--! diketik: validasi real-time saat user mengetik
ketika #email diketik:
  ambil nilai dari #email → simpan ke email
  jika email.panjang > 0 dan bukan email ada di "@":
    perbarui teks "#hint" → "Email tidak valid"

--! ditekan: deteksi Enter untuk submit
ketika #input-tugas ditekan:
  langsung:
    if (event.key === "Enter") {
      document.getElementById("tambah-btn").click();
    }

--! diubah: simpan saat user selesai edit & pindah fokus
ketika #status diubah:
  ambil nilai dari #status → simpan ke status_baru
  --! Simpan ke server

```

◆ CATATAN PENGAJAR — BAB 6

DURASI 3 JAM · TEORI + PRAKTIK BANYAK

TUJUAN BELAJAR

Peserta menguasai 13 keyword event, memahami beda diketik/ditekan/diubah, dan dapat memasang handler untuk berbagai skenario interaksi.

SARAN MENGAJAR

Demo interaktif: form dengan 3 input yang masing-masing pakai diketik, ditekan, diubah. Tunjukkan kapan masing-masing dipicu dengan console.log.

KESALAHAN UMUM

- Tukar **diketik** vs **diubah**
- Lupa **preventDefault** default di **disubmit**
- Pakai **ditekan** padahal butuh **dilepas** (keyup)
- Handler berat di **diketik** → lag

TROUBLESHOOTING

- **E2017**: Target event tidak valid
- **E2018**: Nama event tidak dikenali
- **E4009**: Event name tidak dikenali analyzer

PERTANYAAN DISKUSI

1. Kapan diketik lebih baik dari diubah? 2. Mengapa disubmit default preventDefault? 3. Bagaimana cara deteksi Ctrl+S?

BAB 7

Self-Reference di **ketika**

Fitur baru v0.3.1: **ketika** tanpa target di dalam blok **buat** atau **komponen** otomatis merujuk ke elemen yang sedang dibuat. Ini disebut **self-reference**. Fitur ini

menghilangkan kebutuhan menulis selector berulang — handler langsung menempel ke elemen terdekat dalam AST.

7.1 Sintaks Self-Reference

```
KARSA · 17-SELF-REFERENCE.KS

--! Tanpa self-reference (verbose):
buat tombol#btn-tambah → teks: "Tambah"
ketika #btn-tambah diklik → tambahkan 1 ke counter

--! Dengan self-reference (ergonomis):
buat tombol#btn-tambah → teks: "Tambah"
ketika diklik → tambahkan 1 ke counter
```

Kedua bentuk di atas menghasilkan output kompilasi yang sama. Bedanya hanya gaya penulisan: self-reference lebih ringkas dan tidak perlu mengulang selector. Untuk elemen dengan ID panjang atau nested dalam, self-reference sangat mengurangi boilerplate.

7.2 Aturan Validitas

● HANYA VALID DI BUAT ATAU KOMPONEN

Self-reference **hanya valid** di dalam blok `buat` atau `komponen` yang punya root node tunggal. Di luar konteks ini, `ketika diklik:` tanpa target akan menghasilkan error:

- **E3005:** `ketika` tanpa target hanya boleh di dalam blok `buat` atau `komponen`
- **E5001:** Node AST tidak didukung compiler (saat lowering gagal)

7.3 Contoh Realistis — Todo Item

```
KARSA · 18-TODO-ITEM.KS

--! Setiap item todo punya tombol hapus yang self-reference
ulangi tugas dari semuaTugas:
  buat li
    buat span → teks: tugas
    buat tombol.hapus-btn → teks: "Hapus"
    --! Self-reference: handler menempel ke tombol hapus ini
    ketika diklik:
      --! Hapus tugas ini dari daftar
      langsung:
        const idx = event.target.closest("li").dataset.index;
        semuaTugas.value.splice(idx, 1);
        semuaTugas.value = [...semuaTugas.value];
```

TUJUAN BELAJAR

Peserta memahami self-reference ketika, kapan bisa dipakai, dan kapan harus pakai selector eksplisit.

SARAN MENGAJAR

Demo refactoring: kode lama dengan selector eksplisit → kode baru dengan self-reference. Tunjukkan baris yang berkurang. Lalu trigger E3005 dengan contoh salah.

KESALAHAN UMUM

- Pakai self-reference di top-level → E3005
- Bingung "elemen apa yang dimaksud" saat baca kode
- Lupa bahwa self-reference butuh root tunggal di komponen

TROUBLESHOOTING

- **E3005:** Pindahkan **ketika** ke dalam blok **buat** / **komponen**
- **E5001:** Komponen multi-root tidak dapat self-reference — gunakan **fragmen**

PERTANYAAN DISKUSI

1. Apa keuntungan self-reference selain ringkas? 2. Kapan harus tetap pakai selector eksplisit? 3. Bagaimana self-reference bekerja di komponen multi-root?

BAB 8

Rantai Aksi dengan **lalu**

Kadang Anda perlu mengeksekusi beberapa aksi berurutan dalam satu blok. KARSA menyediakan kata kunci **lalu** untuk merantai aksi — mirip dengan **await** di JavaScript untuk async, atau sekadar urutan eksekusi untuk sync. **lalu** membuat urutan eksplisit tanpa harus menulis baris terpisah.

8.1 Sintaks **lalu**

```
--! Rantai aksi sync
kosongkan input#pesan
  lalu tampilkan notifikasi "Ter kirim!"
  lalu sembunyikan div#loading

--! Setara dengan baris terpisah (sync):
--! kosongkan input#pesan
--! tampilkan notifikasi "Ter kirim!"
--! sembunyikan div#loading
```

8.2 Aturan lalu

● HANYA DI BLOK AKSI, BUKAN AKSI TUNGGAL

`lalu` hanya valid di `blok_aksi` (multi-baris dengan indentasi). Tidak boleh dipakai setelah `→` di aksi tunggal satu baris. Jika butuh sub-blok (seperti `jika` atau `ulangi` di tengah rantai), gunakan baris terpisah.

8.3 Kapan Pakai lalu vs Baris Terpisah

SITUASI	PAKAI LALU	PAKAI BARIS TERPISAH
Aksi singkat berurutan	✓ Cocok	Bisa, tapi verbose
Aksi panjang/berparameter kompleks	Kurang readable	✓ Cocok
Ada sub-blok (<code>jika</code> , <code>ulangi</code>)	Tidak bisa	✓ Wajib
Pipeline async berurutan	✓ Cocok (lihat async)	Bisa pakai <code>setelah</code>

8.4 lalu untuk Async (Pengantar)

Untuk aksi async (seperti `ambil dari URL`), `lalu` berperilaku seperti promise chain — aksi berikutnya menunggu aksi sebelumnya selesai. Detail async akan dibahas di Modul 5.

```
KARSA · 20-LALU-ASYNC.KS (PENGANTAR)

--! Pipeline async dengan lalu
sembunyikan div#konten
  lalu tampilkan div#loading
  lalu ambil dari "/api/data":
    berhasil → simpan hasilnya ke data
    gagal → tampilkan pesan-error "Gagal"
    selalu → sembunyikan div#loading
```

TUJUAN BELAJAR

Peserta memahami `lalu` untuk chaining aksi, batasannya, dan kapan lebih baik pakai baris terpisah.

SARAN MENGAJAR

Demo side-by-side: kode dengan 3 baris terpisah vs 1 rantai `lalu`. Tunjukkan eksekusi identik untuk sync. Bahas async di Modul 5.

KESALAHAN UMUM

- Pakai `lalu` setelah `→` di aksi tunggal
- Merantai aksi berat yang seharusnya pakai sub-blok
- Mengira `lalu` selalu async (sebenarnya sync untuk aksi sync)

TROUBLESHOOTING

- Syntax error: cek `lalu` tidak setelah `→`
- Urutan tidak sesuai: pastikan aksi sync, bukan mixed async

PERTANYAAN DISKUSI

1. Apa beda `lalu` sync vs async? 2. Kapan `lalu` lebih buruk dari baris terpisah? 3. Bisakah `lalu` dipakai di ketika handler?

BAB 9

Lifecycle Hook: `dipasang`, `diperbarui`, `dilepas`

Komponen KARSA memiliki tiga momen lifecycle: ketika pertama kali dipasang ke DOM (`dipasang`), ketika props/state berubah (`diperbarui` — baru di v0.3.1), dan ketika akan dilepas dari DOM (`dilepas`). Hook ini penting untuk manajemen resource: pasang timer di `dipasang` , bersihkan di `dilepas` .

9.1 Tiga Lifecycle Hook

HOOK	KAPAN DIPICU	USE CASE UMUM
saat komponen dipasang:	Node root pertama kali ditambahkan ke DOM	Pasang event listener global, mulai timer, fetch data awal, set fokus awal
saat komponen diperbarui:	Props/state yang terikat berubah (BARU v0.3.1)	Respons perubahan props dari luar, re-render turunan
saat komponen dilepas:	Sebelum node dihapus dari DOM	Cleanup listener, batalkan request, clear timer, mencegah memory leak

9.2 Sintaks Lifecycle

```
KARSA · 21-LIFECYCLE.KS

komponen JamDigital():
  data waktu = "00:00:00"

  --! Lifecycle: dipasang - mulai timer
  saat komponen dipasang:
    langsung:
      const update = () => {
        const now = new Date();
        this.waktu.value = now.toLocaleTimeString();
      };
      update();
      this._timer = setInterval(update, 1000);

  --! Lifecycle: diperbarui - log perubahan
  saat komponen diperbarui:
    tampilkan pesan "Waktu diperbarui."

  --! Lifecycle: dilepas - clear timer
  saat komponen dilepas:
    langsung:
      clearInterval(this._timer);

  buat div.jam -> teks: waktu
```

9.3 Sync dengan Event dipasang/dilepas-dari-dom

Selain lifecycle komponen, ada juga event `dipasang` dan `dilepas-dari-dom` yang dapat dipasang ke elemen biasa via `ketika`. Keduanya sinkron dengan lifecycle komponen — `dipasang` elemen = `dipasang` komponen root.

```
KARSA · 22-EVENT-LIFECYCLE.KS

--! Event lifecycle di elemen biasa (bukan komponen)
buat div#widget
  ketika dipasang:
    tampilkan pesan "Widget aktif!"
    --! Pasang listener global di sini

  ketika dilepas-dari-dom:
    tampilkan pesan "Widget dimatikan."
    --! Cleanup di sini

  buat h3 -> teks: "Widget Saya"
```

9.4 Diagram Lifecycle

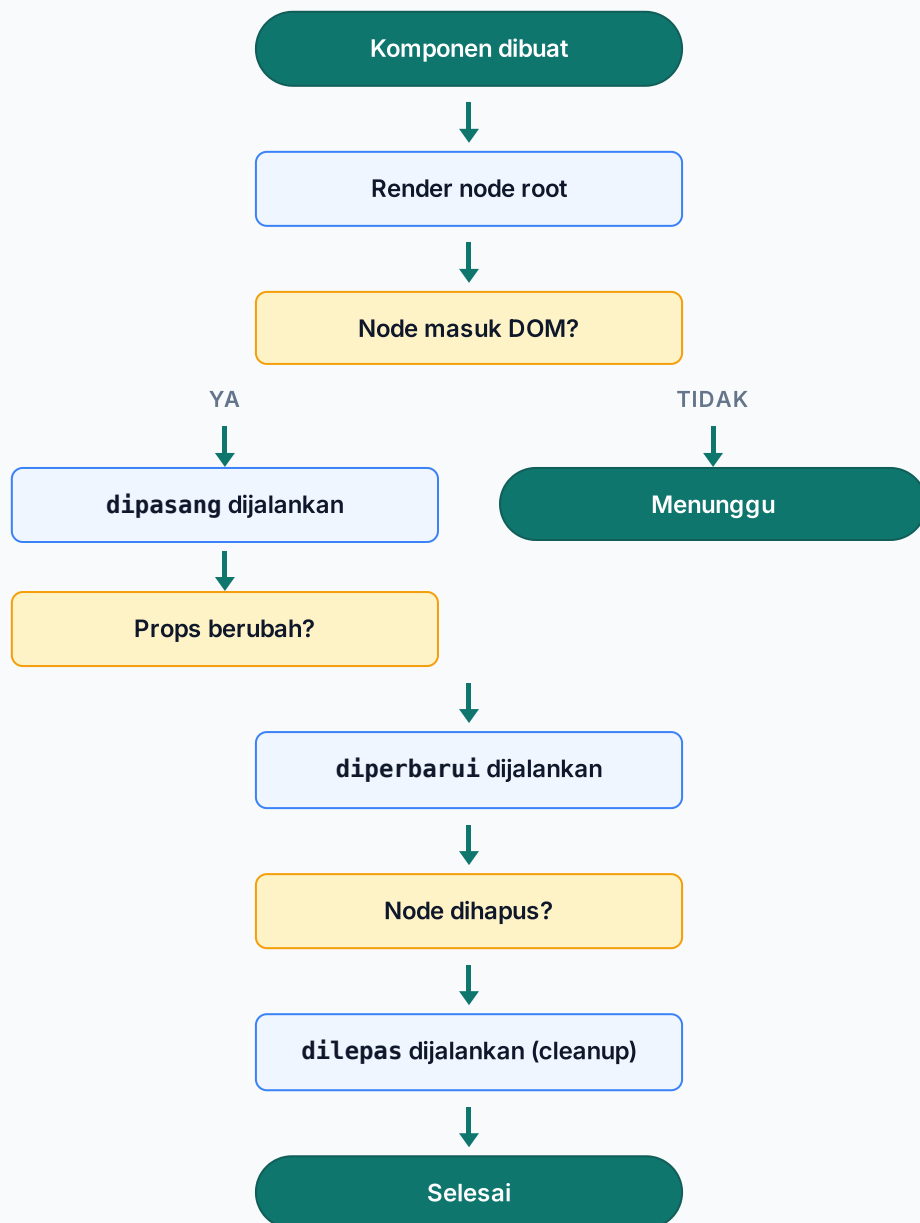


Diagram 3.1 — Alur lifecycle komponen KARS

TUJUAN BELAJAR

Peserta memahami 3 lifecycle hook, kapan dipicu, dan dapat mengelola resource (timer, listener, request) dengan benar.

SARAN MENGAJAR

Demo memory leak: komponen dengan timer yang tidak di-clear. Pasang/hapus berkali-kali, lihat memory naik. Lalu fix dengan dilepas.

KESALAHAN UMUM

- Lupa cleanup di `dilepas` → memory leak
- Side-effect berat di `diperbarui` → loop tak terhingga
- Fetch data di `diperbarui` padahal cukup di `dipasang`

TROUBLESHOOTING

- **E4001**: Lifecycle hook di luar komponen
- **W4002**: Lifecycle di dalam loop/handler (warning)
- Memory leak → cek `dilepas` cleanup

PERTANYAAN DISKUSI

1. Apa beda dipasang di komponen vs di elemen biasa? 2. Kapan diperbarui bahaya? 3. Apa konsekuensi lupa dilepas?

BAB 10

Mini-Proyek: Aplikasi Todo List

Saatnya mengintegrasikan semua yang Anda pelajari di Bab 1–9 menjadi satu aplikasi nyata. Kita akan membangun **Aplikasi Todo List** — diadaptasi dari `example/todo-app.html` di repo dazep01. Aplikasi ini menggabungkan `buat`, selector, `tampilkan`, mutator DOM, event handler, self-reference, dan reaktivitas dalam satu file lengkap.

10.1 Spesifikasi

- **Tambah tugas**: Input teks + tombol "Tambah" → tugas masuk ke daftar.
- **Enter key**: Tekan Enter di input → sama dengan klik tombol Tambah.
- **Hapus tugas**: Setiap item punya tombol "Hapus" → hapus dari daftar.
- **Counter**: Tampilkan total tugas di bagian bawah.
- **Reaktivitas**: Daftar render ulang otomatis setiap kali `semuaTugas` berubah.

10.2 Analisis Alur

KEBUTUHAN	SINTAKS KARSA	ALASAN
State tugas	<code>data semuaTugas = []</code>	Array reaktif untuk multi-item
State input	<code>data inputSekarang = ""</code>	Track nilai input
UI form	<code>buat masukan + buat tombol</code>	Form input sederhana
Event tambah	<code>ketika #tambah-btn diklik</code>	Handler klik tombol
Event Enter	<code>ketika #input-tugas ditekan</code>	Deteksi Enter via keydown
Render list	<code>saat semuaTugas berubah</code>	Reaktivitas: render ulang saat data berubah
Hapus item	Self-reference <code>ketika diklik</code>	Handler menempel ke tombol hapus per item
Counter	<code>perbarui teks #total-angka</code>	Update counter saat list berubah

10.3 Tahap 1 — State & UI Statis

```
KARSA · 23-TODO-TAHAP-1.KS

--! State aplikasi
data semuaTugas = []
data inputSekarang = ""

--! UI statis
buat div.container
  buat h2 → teks: "Daftar Tugasku"

  buat div.input-group
    buat masukan#input-tugas → tipe: "text", placeholder: "Apa yang ingin dikerjakan"
    buat tombol#tambah-btn → teks: "Tambah"

  buat ul#list-tugas

  buat div.info
    buat span → teks: "Total Tugas: "
    buat span#total-angka → teks: "0"
```

10.4 Tahap 2 — Tambah Tugas + Render List

```
KARSA · 24-TODO-TAHAP-2.KS

--! Event: klik tombol Tambah
ketika #tambah-btn diklik:
  ambil nilai dari #input-tugas → simpan ke teksBaru

--! Validasi
jika teksBaru == "":
  tampilkan pesan-error "Tugas tidak boleh kosong."
  berhenti

--! Tambah ke array & reset input
tambahkan teksBaru ke semuaTugas
perbarui nilai #input-tugas → ""

--! Reaktivitas: render ulang saat data berubah
saat semuaTugas berubah:
  --! Bersihkan list
  kosongkan #list-tugas

  --! Tambah ulang semua item
  ulangi tugas dari semuaTugas:
    buat li
      buat span → teks: tugas
      buat tombol.hapus-btn → teks: "Hapus"

  --! Update counter
  perbarui teks #total-angka → semuaTugas.panjang
```

10.5 Tahap 3 — Enter Key + Hapus Tugas

```
KARSA · 25-TODO-TAHAP-3.KS

--! Event: tekan Enter di input
ketika #input-tugas ditekan:
  langsung:
    if (event.key === "Enter") {
      document.getElementById("tambah-btn").click();
    }

--! Event: hapus tugas (dengan self-reference + dataset index)
saat semuaTugas berubah:
  kosongkan #list-tugas

  ulangi tugas dari semuaTugas:
    buat li
      buat span → teks: tugas
      buat tombol.hapus-btn → teks: "Hapus"
      --! Self-reference: handler menempel ke tombol hapus ini
      ketika diklik:
        --! Hapus berdasarkan index (pakai JS untuk splice)
        langsung:
          const idx = event.target.closest("li").dataset.index;
          semuaTugas.value.splice(idx, 1);
          --! Trigger reaktivitas dengan array baru
          semuaTugas.value = [...semaTugas.value];

      --! Injeksi dataset index untuk keperluan hapus
      langsung:
        const items = document.querySelectorAll("#list-tugas li");
        items.forEach((li, i) => li.dataset.index = i);

    perbarui teks #total-angka → semuaTugas.panjang
```

10.6 Versi Final Lengkap

Gabungkan Tahap 1–3 jadi satu file lengkap. Berikut versi final yang siap dipasang ke HTML dengan engine `karsa.standalone.js`:

```

--! State aplikasi
data semuaTugas = []
data inputSekarang = ""

--! UI utama
buat div.container
  buat h2 → teks: "Daftar Tugasku"

  buat div.input-group
    buat masukan#input-tugas → tipe: "text", placeholder: "Apa yang ingin dikerjakan?"
    buat tombol#tambah-btn → teks: "Tambah"

  buat ul#list-tugas

  buat div.info
    buat span → teks: "Total Tugas: "
    buat span#total-angka → teks: "0"

--! Event: tombol Tambah diklik
ketika #tambah-btn diklik:
  ambil nilai dari #input-tugas → simpan ke teksBaru
  jika teksBaru == "":
    tampilkan pesan-error "Tugas tidak boleh kosong."
    berhenti
  tambahkan teksBaru ke semuaTugas
  perbarui nilai #input-tugas → ""

--! Event: Enter di input
ketika #input-tugas ditekan:
  langsung:
    if (event.key === "Enter") {
      document.getElementById("tambah-btn").click();
    }

--! Reaktivitas: render ulang saat data berubah
saat semuaTugas berubah:
  kosongkan #list-tugas
  ulangi tugas dari semuaTugas:
    buat li
      buat span → teks: tugas
      buat tombol.hapus-btn → teks: "Hapus"
      ketika diklik:
        langsung:
          const idx = event.target.closest("li").dataset.index;
          semuaTugas.value.splice(idx, 1);
          semuaTugas.value = [...semaTugas.value];
    langsung:
      const items = document.querySelectorAll("#list-tugas li");

```

```
items.forEach((li, i) => li.dataset.index = i);  
perbarui teks #total-angka -> semuaTugas.panjang
```

10.7 Diskusi Penyempurnaan

- **Filter:** Tambah tab "Semua/Aktif/Selesai" dengan filter array.
- **Persistence:** Simpan ke `localStorage` via `jalankan` saat `semuaTugas` berubah.
- **Edit inline:** Klik dua kali item -> input edit muncul di tempat.
- **Drag-drop:** Reorder dengan HTML5 Drag API via `langsung:`.
- **Animasi:** Tambah transisi CSS untuk item masuk/keluar.

◆ CATATAN PENGAJAR — BAB 10

DURASI 5 JAM • PROYEK MANDIRI

TUJUAN BELAJAR

Peserta mampu mengintegrasikan semua konsep Modul 3 dalam satu aplikasi nyata, dengan state management, event handling, dan reaktivitas yang benar.

SARAN MENGAJAR

Bagi siswa jadi 3 tahap. Tahap 1 selesai dalam 30 menit, code review. Tahap 2 selesai 60 menit, review. Tahap 3 sebagai PR dengan deadline 1 minggu.

KESALAHAN UMUM

- Lupa `kosongkan` sebelum render ulang -> duplikat
- Salah scope `self-reference` di luar `buat`
- Lupa trigger reaktivitas dengan array baru setelah splice
- Validasi input kosong tidak ada -> item kosong masuk list

TROUBLESHOOTING

- Item tidak muncul -> cek `saat semuaTugas berubah`
- Hapus tidak jalan -> cek `dataset.index` terisi
- Enter tidak jalan -> cek `event.key` di `langsung:`

PERTANYAAN DISKUSI

1. Mengapa perlu `[... semuaTugas.value]` setelah splice? 2. Bisakah hapus tanpa `langsung:`? 3. Bagaimana cara persist ke `localStorage`?

LATIHAN

Latihan Soal — 15 Soal Bertingkat

15 soal bertingkat untuk menguji pemahaman Anda tentang DOM & Event di KARSA. Soal dibagi tiga level: **mudah** (sintaks dasar buat/selector/event), **sedang** (kombinasi mutator + handler), dan **sulit** (debugging & optimasi re-render). Coba dahulu sebelum lihat solusi.

A Level Mudah

L1 • MUDAH

MUDAH

Buat struktur DOM KARSa yang setara dengan HTML berikut: `<div id="app" class="container"><h1>Halo</h1></div>`. Gunakan tag alias Indonesia.

L2 • MUDAH

MUDAH

Tulis pernyataan `perbarui` untuk mengubah teks elemen `#counter` menjadi nilai variabel `hitungan`.

L3 • MUDAH

MUDAH

Pasang event handler `ketika` ke tombol `#btn-hapus` yang memanggil `hapus #kartu-1` saat diklik.

L4 • MUDAH

MUDAH

Buat form login dengan dua input (email + password) dan tombol submit. Gunakan tag `frm`, `masukan`, `tombol`.

L5 • MUDAH

MUDAH

Tulis `ambil` untuk membaca nilai input `#email` dan menyimpannya ke variabel `email`.

B Level Sedang

L6 • SEDANG

SEDANG

Buat handler `ketika #btn-toggle diklik` yang toggle visibility elemen `#panel`. Petunjuk: gunakan `langsung:` untuk cek `style.display`.

L7 • SEDANG

SEDANG

Buat list 3 item dengan `buat ul li` bertingkat, masing-masing item punya teks "Item 1", "Item 2", "Item 3". Setiap `li` punya tombol "Hapus" dengan self-reference `ketika diklik`.

L8 • SEDANG

SEDANG

Buat form dengan validasi real-time: `ketika #email diketik`, periksa apakah email mengandung "@". Jika tidak, perbarui teks `#hint` menjadi "Email tidak valid".

L9 · SEDANG

SEDANG

Buat handler `ketika #input ditekan` yang deteksi tombol Enter (Enter key) lalu klik tombol `#submit`. Gunakan `langsung:` untuk akses `event.key`.

L10 · SEDANG

SEDANG

Buat komponen `KartuProfil(nama, peran)` sederhana dengan struktur: `div.kartu` berisi `h3` (nama) dan `p` (peran). Tampilkan dengan `gunakan` ke `#app`.

c Level Sulit

L11 · SULIT

SULIT

Debug kode berikut. Apa yang salah?

```
buat tombol#btn → teks: "Klik"
ketika diklik → tampilkan pesan "Halo!"
```

Mengapa error E3005 muncul dan bagaimana cara memperbaikinya?

L12 · SULIT

SULIT

Identifikasi anti-pattern pada kode berikut. Apa risikonya? Tulis ulang yang aman.

```
ketika #search diketik:
  ambil nilai dari #search → simpan ke q
  ambil dari "/api/search?q=" + q:
    berhasil → tampilkan hasilnya di "#results"
```

L13 · SULIT

SULIT

Refaktor kode berikut untuk pakai self-reference. Jelaskan keuntungan versi baru.

```
buat tombol#btn-tambah → teks: "Tambah"
buat tombol#btn-reset → teks: "Reset"
ketika #btn-tambah diklik → tambahkan 1 ke counter
ketika #btn-reset diklik → simpan 0 ke counter
```

L14 · SULIT

SULIT

Optimasi re-render: kode berikut me-render ulang 1000 item setiap perubahan. Bagaimana cara mengurangi overhead?

```
saat semuaItem berubah:
  kosongkan #list
  ulangi item dari semuaItem:
    buat li → teks: item
```

L15 · SULIT

SULIT

Buat komponen `LiveClock()` yang menampilkan waktu saat ini dan update setiap detik. Pastikan timer dibersihkan di `dilepas` untuk mencegah memory leak. Petunjuk: gunakan `langsung:` untuk `setInterval` dan `clearInterval`.

SOLUSI

Solusi Latihan Lengkap

Berikut solusi untuk semua 15 soal. Format: kode solusi, penjelasan singkat, alternatif jawaban, dan pembahasan error umum.

SOLUSI L1

MUDAH

```
buat ruang#app.container
  buat judul → teks: "Halo"
```

Penjelasan: ruang adalah alias untuk div, judul untuk h1. Indentasi 2 spasi menandakan judul adalah child dari ruang#app. **Alternatif:** buat div#app.container + buat h1 juga valid. **Error umum:** lupa indentasi → h1 jadi sibling, bukan child.

SOLUSI L2

MUDAH

```
perbarui teks "#counter" → hitungan
```

Penjelasan: Sintaks perbarui `<properti> <target> -> <nilai>`. teks dipetakan ke `innerText`. **Alternatif:** perbarui teks `#counter -> hitungan` (tanpa kutip). **Error umum:** salah urutan, lupa `->`.

SOLUSI L3

MUDAH

```
ketika #btn-hapus diklik → hapus "#kartu-1"
```

Penjelasan: Bentuk aksi tunggal dengan `->`. hapus memanggil `.remove()` — elemen hilang dari DOM. **Alternatif:** blok aksi jika butuh multi-aksi. **Error umum:** lupa `#` di ID.

SOLUSI L4

MUDAH

```
buat frm#login
  buat masukan#email → tipe: "email", placeholder: "Email"
  buat masukan#password → tipe: "password", placeholder: "Password"
  buat tombol → teks: "Login"
```

Penjelasan: Tag frm=form, masukan=input, tombol=button. Properti inline dengan koma. **Alternatif:** form, input, button native juga valid. **Error umum:** lupa koma antar properti.

SOLUSI L5

MUDAH

```
ambil nilai dari "#email" → simpan ke email
```

Penjelasan: Sintaks wajib ambil <jenis> dari <sumber> -> simpan ke <variabel>. nilai dipetakan ke .value. **Alternatif:** tanpa kutip #email juga valid. **Error umum:** lupa -> simpan ke.

SOLUSI L6

SEDANG

```
ketika #btn-toggle diklik:
  langsung:
    const panel = document.querySelector("#panel");
    if (panel.style.display === "none") {
      panel.style.display = "block";
    } else {
      panel.style.display = "none";
    }
  }
```

Penjelasan: KARSA punya sembunyikan tapi tidak ada "tampilkan" untuk display:none. Pakai langsung: dengan cek kondisi. **Alternatif:** track state di variabel data terlihat = benar lalu if/else. **Error umum:** lupa reset display ke "block" (bukan "").

SOLUSI L7

SEDANG

```
buat ul#list
  buat li
    buat span → teks: "Item 1"
    buat tombol → teks: "Hapus"
    ketika diklik → tampilkan pesan "Hapus item 1"
  buat li
    buat span → teks: "Item 2"
    buat tombol → teks: "Hapus"
    ketika diklik → tampilkan pesan "Hapus item 2"
  buat li
    buat span → teks: "Item 3"
    buat tombol → teks: "Hapus"
    ketika diklik → tampilkan pesan "Hapus item 3"
```

Penjelasan: Self-reference ketika diklik menempel ke tombol terdekat dalam blok buat.

Indentasi 4 spasi = child li, 6 spasi = child tombol. **Alternatif:** pakai ulangi dengan array. **Error**

umum: lupa indentasi → handler tidak menempel.

SOLUSI L8

SEDANG

```
ketika #email diketik:
  ambil nilai dari "#email" → simpan ke email
  --! Cek apakah mengandung "@"
  langsung:
    if (!email.value.includes("@")) {
      document.querySelector("#hint").innerText = "Email tidak valid";
    } else {
      document.querySelector("#hint").innerText = "";
    }
  }
```

Penjelasan: diketik = event input, dipicu setiap karakter berubah. langsung: untuk akses includes() karena KARSA tidak punya operator "ada di" string native. **Alternatif:** pakai jalankan ke helper JS. **Error umum:** pakai diubah padahal butuh real-time.

SOLUSI L9

SEDANG

```
ketika #input ditekan:
  langsung:
    if (event.key === "Enter") {
      document.querySelector("#submit").click();
    }
```

Penjelasan: ditekan = keydown, cocok untuk shortcut keyboard. event . key berisi nama tombol.

Alternatif: cek event.keyCode === 13 (legacy). **Error umum:** pakai dilepas (keyup) — biasanya tidak masalah tapi sedikit delay.

SOLUSI L10

SEDANG

```
komponen KartuProfil(nama: teks, peran: teks):
  buat div.kartu
    buat h3 → teks: nama
    buat p → teks: peran

gunakan KartuProfil dengan nama: "Budi", peran: "Admin" di "#app"
```

Penjelasan: Komponen wajib PascalCase. Parameter bisa type hint. gunakan untuk invoke dengan props. **Alternatif:** tanpa type hint juga valid. **Error umum:** nama komponen lowercase → E2003.

SOLUSI L11

SULIT

```
--! KODE ASLI (SALAH):
--! "ketika diklik" tanpa target di top-level = E3005

--! PERBAIKAN: pindahkan ketika ke dalam blok buat
buat tombol#btn → teks: "Klik"
  ketika diklik → tampilkan pesan "Halo!"

--! ATAU: pakai selector eksplisit
buat tombol#btn → teks: "Klik"
ketika #btn diklik → tampilkan pesan "Halo!"
```

Penjelasan: Self-reference hanya valid di blok buat/komponen. Di top-level, harus ada selector eksplisit. **Error umum:** mengira ketika diklik selalu valid — padahal butuh konteks elemen.

SOLUSI L12

SULIT

KARSA · SOLUSI-L12.KS

```
--! ANTI-PATTERN: fetch setiap ketukan → DDoS server
--! Risk: 100+ request/detik, lag, blocking

--! FIX: debounce 300ms
data timer_search = kosong

ketika #search diketik:
  ambil nilai dari "#search" → simpan ke q
  langsung:
    clearTimeout(timer_search.value);
    timer_search.value = setTimeout(() ⇒ {
      --! fetch hanya setelah user berhenti ngetik 300ms
      console.log("Search:", "/api/search?q=" + q.value);
    }, 300);
```

Penjelasan: diketik dipicu setiap karakter. Tanpa debounce, fetch ke server setiap ketukan = DDoS. setTimeout dengan clearTimeout menunda fetch sampai user berhenti mengetik 300ms.

Alternatif: pakai diubah (blur-triggered) jika real-time tidak krusial.

SOLUSI L13

SULIT

KARSA · SOLUSI-L13.KS

```
--! REFACTOR dengan self-reference
buat tombol#btn-tambah → teks: "Tambah"
  ketika diklik → tambahkan 1 ke counter

buat tombol#btn-reset → teks: "Reset"
  ketika diklik → simpan 0 ke counter
```

Penjelasan: Self-reference menghilangkan kebutuhan selector eksplisit #btn-tambah. Handler otomatis menempel ke tombol terdekat. **Keuntungan:** (1) lebih ringkas, (2) jika ganti ID, handler tetap menempel, (3) intent "tombol ini diklik" lebih jelas.

```
--! OPTIMASI 1: Hanya render item baru/berubah, bukan semua
--! Gunakan key tracking dengan data-id
```

```
--! OPTIMASI 2: DocumentFragment untuk batch insert
```

saat semuaItem berubah:

langsung:

```
const frag = document.createDocumentFragment();
semuaItem.value.forEach((item) => {
  const li = document.createElement("li");
  li.innerText = item;
  frag.appendChild(li);
});
const list = document.querySelector("#list");
list.replaceChildren(frag); --! 1 reflow, bukan 1000
```

Penjelasan: (1) DocumentFragment menggabungkan 1000 appendChild jadi 1 operasi — browser hanya reflow 1 kali, bukan 1000. (2) Idealnya pakai *diffing* seperti virtual DOM, tapi KARSA tidak punya built-in. Untuk list <100 item, render ulang semua masih OK. **Alternatif:** pagination — hanya render 50 item per halaman.

```
komponen LiveClock():  
  data waktu = "00:00:00"  
  tetap _timer = kosong  
  
  saat komponen dipasang:  
    langsung:  
      const update = () => {  
        const now = new Date();  
        waktu.value = now.toLocaleTimeString();  
      };  
      update(); --! set langsung  
      _timer.value = setInterval(update, 1000);  
  
  saat komponen dilepas:  
    langsung:  
      clearInterval(_timer.value); --! cleanup!  
  
  buat div.jam → teks: waktu
```

Penjelasan: dipasang menjalankan timer, dilepas membersihkan. Tanpa `clearInterval`, timer tetap jalan setelah komponen dihapus → memory leak. **Alternatif:** pakai `requestAnimationFrame` untuk akurasi tinggi. **Test:** pasang/hapus komponen 100x di DevTools, cek memory tidak naik.

LAMPIRAN A

Referensi Cepat

Cheat sheet satu halaman berisi semua kata kunci Modul 3.

Pembuatan Elemen

```
--! buat elemen  
buat tag#id.kelas[attr]  
  teks: "..."  
  buat child
```

Mutasi DOM

```
perbarui teks|html|nilai|kelas|gaya <target> → <nilai>  
sembunyikan <target>  
hapus <target>  
kosongkan <target>
```

Output

```
--! tampilkan
tampilkan <el> di "#target"
dengan mode: "ganti"

--! 5 mode:
--! tambahkan/ganti/awalan/
--! sebelum/sesudah

--! pesan
tampilkan pesan "..."
tampilkan pesan-error "..."
tampilkan notifikasi "..."
```

Ambil (Read DOM)

```
ambil nilai|teks|html|tinggi|lebar
dari <sumber> → simpan ke <var>
ambil atribut "x" dari <src> → simpan ke <var>
```

Event & Lifecycle

```
--! event handler
ketika <target> <event>:
    <aksi>

--! 13 event:
--! diklik/diketik/disubmit/
--! dimuat/diubah/difokus/
--! ditinggal/ditekan/dilepas/
--! diarahkan/ditinggal-kursor/
--! digulir/dipasang/
--! dilepas-dari-dom

--! lifecycle komponen
saat komponen dipasang:
saat komponen diperbarui:
saat komponen dilepas:

--! chaining
<aksi> lalu <aksi> lalu <aksi>
```

LAMPIRAN B

KARSA vs Vanilla JS DOM API

Tabel padanan sintaks KARSA dengan vanilla JavaScript DOM API. Berguna untuk developer yang sudah familiar JavaScript dan sedang transisi.

OPERASI	KARSA	VANILLA JS
Buat elemen	buat div#app	<code>document.createElement("div"); el.id="app"</code>
Append child	buat div (dalam parent)	<code>parent.appendChild(el)</code>
Set teks	→ teks: "Halo"	<code>el.innerText = "Halo"</code>
Set HTML	→ html: "..."	<code>el.innerHTML = "..."</code>
Set kelas	→ kelas: "aktif"	<code>el.className = "aktif"</code>
Set atribut	→ src: "/img.jpg"	<code>el.setAttribute("src", "/img.jpg")</code>
Query selector	"#id" atau div#id	<code>document.querySelector("#id")</code>
Tampilkan (append)	tampilkan el di "#target"	<code>target.appendChild(el)</code>
Tampilkan (replace)	tampilkan el di "#t" dengan mode: "ganti"	<code>target.replaceChildren(el)</code>
Tampilkan (prepend)	... dengan mode: "awalan"	<code>target.prepend(el)</code>
Tampilkan (before)	... dengan mode: "sebelum"	<code>target.before(el)</code>
Tampilkan (after)	... dengan mode: "sesudah"	<code>target.after(el)</code>
Update teks	perbarui teks "#id" → nilai	<code>el.innerText = nilai</code>
Sembunyikan	sembunyikan "#id"	<code>el.style.display = "none"</code>
Hapus	hapus "#id"	<code>el.remove()</code>
Kosongkan child	kosongkan "#id"	<code>el.replaceChildren()</code>
Baca nilai	ambil nilai dari "#id" → simpan ke x	<code>let x = el.value</code>
Baca atribut	ambil atribut "data-id" dari "#id" → simpan ke x	<code>let x = el.getAttribute("data- id")</code>
Event click	ketika #btn diklik: ...	<code>btn.addEventListener("click", ...)</code>
Event input	ketika #input diketik: ...	<code>input.addEventListener("input", ...)</code>

OPERASI	KARSA	VANILLA JS
Event submit	ketika form disubmit: ...	form.addEventListener("submit", e ⇒ { e.preventDefault(); ... })
Event keydown	ketika #input ditekan: ...	input.addEventListener("keydown", ...)

LAMPIRAN C

Tabel 20+ Tag Alias Indonesia

Tabel lengkap tag alias Indonesia yang dikenali parser KARSA. Alias ini dapat dipakai sebagai pengganti tag HTML native di pernyataan `buat` dan selector.

ALIAS INDONESIA	TAG HTML	KEGUNAAN & CATATAN
ruang	div	Wadah blok generik — paling sering dipakai
wadah	div	Sinonim <code>ruang</code> , untuk variasi semantik
kotak	div	Wadah dengan konotasi "box" (bergaya)
wadjud	div	Wadah khusus (custom, jarang dipakai)
judul	h1	Judul utama halaman (satu per halaman)
subjudul	h2	Sub-judul section
paragraf	p	Paragraf teks
gambar	img	Gambar — wajib <code>src</code> , disarankan <code>alt</code>
tautan	a	Hyperlink — wajib <code>href</code>
masukan	input	Field input form — wajib <code>type</code>
kolom	textarea	Input teks multi-baris
tombol	button	Tombol yang dapat diklik
frm	form	Formulir
frmMasuk	form	Form khusus login (custom)
tabel	table	Tabel data
artikel	article	Konten artikel mandiri
kanvas	canvas	Kanvas untuk gambar 2D via JS
opsi	option	Pilihan dalam dropdown <code>pilihan</code>
pilihan	select	Dropdown pilihan — berisi <code>opsi</code>
fragmen	DocumentFragment	Wadah virtual, tidak render langsung

● CATATAN PENGGUNAAN

Tag HTML native (`div`, `span`, `h1` - `h6`, `header`, `footer`, `main`, `section`, `nav`, `aside`, `video`, `audio`, `label`, dll) juga valid di KARSA. Alias Indonesia hanyalah alternatif — keduanya dapat dipakai berdampingan dalam file yang sama, meskipun konsistensi direkomendasikan.

Error Codes DOM & Event

Daftar error codes yang relevan untuk Modul 3 (DOM & Event). Mencakup Lexer (E1xxx), Parser (E2xxx), Resolver (E3xxx), Analyzer (E4xxx), dan Compiler (E5xxx).

KODE	SEVERITY	STAGE	PESAN	SARAN
E1009	ERROR	Lexer	Selector CSS tidak valid	Pastikan selector CSS valid (#id, .class, tag)
E2001	ERROR	Parser	Diharapkan <expected>, tetapi ditemukan <actual>	Periksa sintaksis pada lokasi yang ditunjuk
E2002	ERROR	Parser	Selector tidak valid	Diawali nama tag HTML atau identifier
E2015	ERROR	Parser	Selector CSS tidak valid (lanjut)	Periksa konteks penggunaan selector
E2016	ERROR	Parser	Token '→' diharapkan	Gunakan pola: perbarui <prop> <target> → <nilai>
E2017	ERROR	Parser	Target event tidak valid	Periksa target dan nama event
E2018	ERROR	Parser	Nama event tidak valid	Pakai: diklik, diketik, disubmit, dll
E2022	ERROR	Parser	Target tampilkan tidak valid	Periksa target tampilkan
E2024	ERROR	Parser	'ambil' tanpa konteks yang jelas	Gunakan: ambil <jenis> dari <sumber> → simpan ke <nama>
E2027	ERROR	Parser	Properti perbarui tidak dikenali	Pakai: teks, html, kelas, src, href, dll
E3005	ERROR	Resolver	'ketika' tanpa target hanya boleh di blok 'buat'/'komponen'	Tambahkan target atau pindahkan ke blok buat
E4007	ERROR	Analyzer	Mode tampilkan tidak valid	Mode valid: tambahkan, ganti, awalan, sebelum, sesudah
E4008	ERROR	Analyzer	Properti perbarui tidak didukung	Pakai properti yang didukung oleh perbarui
E4009	ERROR	Analyzer	Event name tidak dikenali	Event valid: diklik, diketik, ditekan, dll
E4010	ERROR	Analyzer	Penggunaan 'gunakan' untuk non-komponen	Pastikan nama PascalCase adalah komponen
E5001	ERROR	Compiler	Node AST tidak didukung compiler	Periksa apakah node type sudah didukung

KODE	SEVERITY	STAGE	PESAN	SARAN
E5003	ERROR	Compiler	Selector tidak dapat dikompilasi	Periksa format selector CSS
W2002	WARNING	Parser	Blok kosong terdeteksi	Tambahkan setidaknya satu statement
W4002	WARNING	Analyzer	Lifecycle hook di dalam loop/handler	Pindahkan lifecycle ke level komponen

Selamat!

Anda telah menyelesaikan Modul 3 KARSA

Anda kini menguasai pembangunan antarmuka interaktif dengan KARSA — dari menciptakan elemen, memilih dengan selector, memasang event handler, hingga lifecycle komponen. Ini adalah fondasi yang akan dipakai di seluruh aplikasi web yang Anda bangun.

YANG TELAH ANDA PELAJARI

- ✓ Bab 1 — **buat** & 20+ tag alias
- ✓ Bab 2 — Selector **tag#id.kelas[attr]**
- ✓ Bab 3 — 13 properti + 6 jenis **ambil**
- ✓ Bab 4 — **tampilkan** 5 mode
- ✓ Bab 5 — **perbarui** / **sembunyikan** / **hapus** / **kosongkan**
- ✓ Bab 6 — 13 keyword event **ketika**
- ✓ Bab 7 — Self-reference **ketika**
- ✓ Bab 8 — Rantai aksi **lalu**
- ✓ Bab 9 — Lifecycle **dipasang** / **diperbarui** / **dilepas**
- ✓ Bab 10 — Mini-proyek Todo List

"Antarmuka yang baik bukan yang punya banyak fitur, tapi yang membuat pengguna merasa pintar."